

Résumé du code — Projet Stargate

1. Variables globales — FormTdB

Le tableau de bord utilise un **Singleton** pour la connexion SQLite et un **DataSet global partagé** (MesDatas.DsGlobal) entre tous les formulaires.

■ Classe Connexion — Singleton

```
private static SQLiteConnection connec;  
public static SQLiteConnection Connec {  
    get {  
        if (connec == null) {  
            connec = new SQLiteConnection(@"Data Source = Stargate.db");  
            connec.Open();  
        }  
        return connec;  
    }  
}
```

■ Classe MesDatas — DataSet global

```
public static DataSet DsGlobal = new DataSet();  
// Chargé une fois au démarrage, consulté en mode déconnecté partout
```

■ Variables de classe FormTdB

```
private static readonly Color CouleurSurvol = Color.FromArgb(224, 238, 249);  
private static readonly Color CouleurNormale = Color.White;  
private string[] _derniersFiltres = null; // mémorise l'état des filtres
```

2. Code complet du tableau de bord — FormTdB

Le tableau de bord charge toutes les tables en mode **connecté** au démarrage, puis affiche les missions via des **UserControl** créés dynamiquement. Les filtres fonctionnent en mode **déconnecté**.

■ Chargement de toutes les tables (mode connecté)

```
private void ChargerDonneesBDD()
{
    SQLiteConnection conn = Connexion.Connec;
    DataTable schema = conn.GetSchema("Tables");
    foreach (DataRow row in schema.Rows)
    {
        string nomTable = row[2].ToString();
        string sql = "SELECT * FROM " + nomTable;
        SQLiteDataAdapter da = new SQLiteDataAdapter(sql, conn);
        if (MesDatas.DsGlobal.Tables.Contains(nomTable))
            MesDatas.DsGlobal.Tables[nomTable].Clear();
        da.Fill(MesDatas.DsGlobal, nomTable);
    }
}
```

■ Affichage des missions — UserControls dynamiques

```
private void ChargerMissions()
{
    int marge=20, espY=15, ucHauteur=215, y=marge, count=0;
    int ucLargeur = panel1.ClientSize.Width - marge*2
    - SystemInformation.VerticalScrollBarWidth;
    foreach (DataRow row in MesDatas.DsGlobal.Tables["mission"].Rows)
    {
        DataRow[] dr = MesDatas.DsGlobal.Tables["membre"]
        .Select("matricule = '" + row[5] + "'");
        if (dr.Length == 0) continue;
        UserControl1 uc = new UserControl1(
            row[0]+row[1].ToString(), row[3].ToString(), row[4].ToString(),
            dr[0][1]+" "+dr[0][2], "Planetes/Logo - "+row[0]+".png",
            row[8].ToString(), row[3].ToString(), row[4].ToString());
        uc.setNomPlanete(row[0].ToString());
        uc.setNumeroMission(Convert.ToInt32(row[1]));
        uc.OuvrirFormulaire += UserControl1_OuvrirFormulaire;
        panel1.Controls.Add(uc);
        y += ucHauteur + espY; count++;
    }
}
```

■ Filtrage avec opérateurs de comparaison

```

private void AppliquerFiltres()
{
    string rechercheChef = textBox1.Text.Trim().ToLower();
    string planete = comboBox1.SelectedIndex>0 ? comboBox1.SelectedItem.ToString() : "";
    string opBudget = comboBox4.SelectedItem?.ToString() ?? "";
    foreach (DataRow row in MesDatas.DsGlobal.Tables["mission"].Rows)
    {
        if (!string.IsNullOrEmpty(planete) && row[0].ToString() != planete) continue;
        if (filtererBudget && !CompareValeur(budget, opBudget, valBudget)) continue;
        panel1.Controls.Add(new UserControl1(...));
    }
}

private bool CompareValeur(int valeur, string op, int reference)
{
    switch (op) {
        case "<": return valeur < reference;
        case "<=": return valeur <= reference;
        case "=": return valeur == reference;
        case ">=": return valeur >= reference;
        case ">": return valeur > reference;
        default: return true;
    }
}

```

■ Ouverture du résumé et rafraîchissement

```

private void UserControl1_OuvrirFormulaire(object sender, EventArgs e)
{
    UserControl1 uc = (UserControl1)sender;
    new FormResumeMission(uc.getNomPlanete(), uc.getNumeroMission()).Show();
}

public void Rafraichir()
{
    // Recharge tout le DataSet puis recrée les UserControls
    foreach (DataRow row in Connexion.Connec.GetSchema("Tables").Rows) {
        string t = row[2].ToString();
        MesDatas.DsGlobal.Tables[t].Clear();
        new SQLiteDataAdapter("SELECT * FROM "+t, Connexion.Connec).Fill(MesDatas.DsGlobal, t);
    }
    panel1.Controls.Clear(); ChargerMissions();
}

```

3. UserControl le plus intéressant — Form_Aliens (Volet 4)

Form_Aliens combine : cache d'images, création dynamique de cartes avec effets hover, filtrage LINQ en mode déconnecté, et FlowLayoutPanel double-bufferisé.

■ Cache d'images et FlowLayoutPanel double-bufferisé

```
private Dictionary<string,Image> _cacheImages = new Dictionary<string,Image>();
private Image ChargerImage(string chemin)
{
    if (_cacheImages.ContainsKey(chemin)) return _cacheImages[chemin];
    if (File.Exists(chemin)) {
        Image img = Image.FromFile(chemin);
        _cacheImages[chemin] = img;
        return img;
    }
    return null;
}

private class FlowLayoutPanelDB : FlowLayoutPanel {
    public FlowLayoutPanelDB() {
        this.DoubleBuffered = true;
        this.SetStyle(ControlStyles.OptimizedDoubleBuffer |
            ControlStyles.AllPaintingInWmPaint, true);
    }
}
```

■ Filtrage en mode déconnecté avec LINQ

```
private void AppliquerFiltres()
{
    string nom = textBox1.Text.Trim().ToLower();
    string couleur = comboBox1.SelectedItem?.ToString() ?? "(Toutes)";
    string type = comboBox2.SelectedItem?.ToString() ?? "(Tous)";
    var lignes = _ds.Tables["Races"].AsEnumerable().Where(r =>
        r["nom"].ToString().ToLower().Contains(nom)
        && (couleur=="(Toutes)" || r["couleur"].ToString()==couleur)
        && (type=="(Tous)" || r["type"].ToString()==type)
    ).ToList();
    flowRaces.SuspendLayout();
    flowRaces.Controls.Clear();
    foreach (var row in lignes) flowRaces.Controls.Add(CreerCarteRace(row));
    flowRaces.ResumeLayout();
}
```

■ Création dynamique d'une carte avec effets hover

```

private Panel CreerCarteRace(DataRow row)
{
    PanelDB carte = new PanelDB { Size=new Size(200,270), BackColor=Color.White };
    Pen pen = allie ? _penAllie : ennemi ? _penEnnemi : _penInc;
    carte.Paint += (s,e) => e.Graphics.DrawRectangle(pen,1,1,carte.Width-3,carte.Height-3);
    img.Image = ChargerImage(Path.Combine(_cheminImages, row["nom"]+".png"));
    Action colorerHover = () => { carte.BackColor=_couleurHover; };
    Action colorerNormal = () => { carte.BackColor=Color.White; };
    carte.MouseEnter += (s,e) => colorerHover();
    carte.MouseLeave += (s,e) => {
        if (!carte.ClientRectangle.Contains(carte.PointToClient(Cursor.Position)))
        colorerNormal();
    };
    EventHandler clic = (s,e) => AfficherDetail(row);
    carte.Click += clic; img.Click += clic;
    return carte;
}

```

3bis. UserControl1 — Carte de mission

UserControl1 est le composant visuel de chaque mission. Il utilise **OnPaint** pour un fond semi-transparent, un badge d'état arrondi via **GraphicsPath**, et un événement **OuvrirFormulaire** vers le tableau de bord.

■ Variables — couleurs thème spatial

```
public event EventHandler OuvrirFormulaire;
private string NomPlanete;
private int NumeroMission;
private static readonly Color CouleurBordure = Color.FromArgb(255, 30, 110, 220);
private static readonly Color CouleurAVenir = Color.FromArgb(255, 30, 120, 200); // bleu
private static readonly Color CouleurEnCours = Color.FromArgb(255, 20, 160, 80); // vert
private static readonly Color CouleurTermine = Color.FromArgb(255, 120, 120, 120); // gris
```

■ Constructeur — calcul durée et état

```
public UserControl1(string nom_mission, string date_dep, string date_fin, ...)
{
    label1.Text = "Mission " + nom_mission;
    if (DateTime.TryParse(date_dep_raw, out DateTime dep) &&
        DateTime.TryParse(date_fin_raw, out DateTime fin))
    {
        int nbJours = (int)(fin - dep).TotalDays;
        DateTime today = DateTime.Today;
        if (today < dep) { label7.Text="À venir"; label7.BackColor=CouleurAVenir; }
        else if (today >= dep && today <= fin) { label7.Text="En cours";
            label7.BackColor=CouleurEnCours; }
        else { label7.Text="Terminée"; label7.BackColor=CouleurTermine; }
    }
    try { pictureBox1.Image = Image.FromFile(image_path); } catch { }
    AppliquerStyle();
}
```

■ OnPaint — fond semi-transparent et bordure

```
protected override void OnPaint(PaintEventArgs e)
{
    Graphics g = e.Graphics;
    g.SmoothingMode = SmoothingMode.AntiAlias;
    using (SolidBrush b = new SolidBrush(Color.FromArgb(200, 10, 18, 38)))
    g.FillRectangle(b, new Rectangle(0,0,Width-1,Height-1));
    using (SolidBrush b = new SolidBrush(CouleurBordure))
    g.FillRectangle(b, new Rectangle(0,0,4,Height)); // barre gauche
    using (Pen p = new Pen(Color.FromArgb(100,30,110,220),1))
    g.DrawRectangle(p, new Rectangle(0,0,Width-1,Height-1));
    base.OnPaint(e);
}
```

■ Badge état arrondi via GraphicsPath

```

label7.Paint += (s, pe) =>
{
    pe.Graphics.SmoothingMode = SmoothingMode.AntiAlias;
    using (SolidBrush b = new SolidBrush(label7.BackColor))
    {
        var path = new GraphicsPath();
        int r = 8;
        path.AddArc(0, 0, r,r, 180, 90);
        path.AddArc(label7.Width-r, 0, r,r, 270, 90);
        path.AddArc(label7.Width-r, label7.Height-r,r,r, 0, 90);
        path.AddArc(0, label7.Height-r,r,r, 90, 90);
        path.CloseFigure();
        pe.Graphics.FillPath(b, path);
    }
    TextRenderer.DrawText(pe.Graphics, label7.Text, label7.Font,
        label7.ClientRectangle, Color.White,
        TextFormatFlags.HorizontalCenter | TextFormatFlags.VerticalCenter);
};

```

■ Événement OuvrirFormulaire

```

private void button1_Click(object sender, EventArgs e)
{
    OuvrirFormulaire?.Invoke(this, EventArgs.Empty);
    // Abonnement dans FormTdB : uc.OuvrirFormulaire += UserControl1_OuvrirFormulaire;
}

```

4. Extrait — Génération du PDF (FormJdB)

Le PDF est généré avec **iTextSharp**. Un **PageEvent** dessine le fond et la décoration de chaque page. Des helpers factorisent le style.

■ PageEvent — décoration de chaque page

```
public class StargatePdfPageEvent : PdfPageEventHelper
{
    public override void OnEndPage(PdfWriter writer, Document document)
    {
        PdfContentByte cb = writer.DirectContentUnder;
        cb.SetColorFill(NOIR);
        cb.Rectangle(0,0,document.PageSize.Width,document.PageSize.Height); cb.Fill();
        cb.SetColorStroke(CYAN); cb.SetLineWidth(1.2f);
        cb.Rectangle(15,15,document.PageSize.Width-30,document.PageSize.Height-30);
        cb.Stroke();
        DrawCorner(cb,15,15,OR,0);
        cb.BeginText();
        cb.ShowTextAligned(ALIGN_CENTER,$"PAGE {writer.PageNumber} // SGC",
            document.PageSize.Width/2, 20, 0);
        cb.EndText();
    }
}
```

■ LancerPDF — structure principale

```
private void LancerPDF()
{
    Document doc = new Document(PageSize.A4, 30, 30, 40, 40);
    string filePath = $"Rapport_{this.planete}_{this.num}.pdf";
    PdfWriter writer = PdfWriter.GetInstance(doc, new FileStream(filePath,
        FileMode.Create));
    writer.PageEvent = new StargatePdfPageEvent(NOIR, CYAN, OR);
    doc.Open();
    // Tableau infos mission
    PdfPTable infoTbl = new PdfPTable(new float[] { 1f, 2f });
    Action<string,string> addRow = (label, value) => {
        PdfPCell lbl = new PdfPCell(new Phrase(label, fBold));
        lbl.Border = Rectangle.LEFT_BORDER; lbl.BorderColorLeft = CYAN;
        infoTbl.AddCell(lbl);
        infoTbl.AddCell(new PdfPCell(new Phrase(value, fNormal)));
    };
    addRow("PLANETE", this.planete); addRow("COMMANDANT", chef);
    doc.Add(infoTbl); doc.Close();
}
```

■ Helper SectionHeader

```
private static PdfPTable SectionHeader(string texte, Font font, BaseColor couleur)
{
    PdfPTable t = new PdfPTable(1);
    PdfPCell cell = new PdfPCell(new Phrase(texte, font));
    cell.BackgroundColor = new BaseColor(20, 30, 45);
    cell.Border = Rectangle.LEFT_BORDER | Rectangle.BOTTOM_BORDER;
    cell.BorderColorLeft = couleur; cell.BorderWidthLeft = 4f;
    t.AddCell(cell);
return t;
}
```

5. Navigation en liaison de données — Événements du journal

Les données sont chargées **une seule fois** dans un DataTable local, puis on navigue avec un simple **index entier**. Aucun accès BDD lors des clics.

■ Chargement unique dans un DataTable local

```
private DataTable dtEvenements;
private int indexEvenement = 0;
private void ChargerEvenements()
{
    string filtre = $"nomPlanete = '{this.planete}' AND numero = {this.num}";
    DataRow[] rows = MesDatas.DsGlobal.Tables["JournalDeBord"]
        .Select(filtre, "dateJ ASC");
    dtEvenements = new DataTable();
    dtEvenements.Columns.Add("dateJ");
    dtEvenements.Columns.Add("commentaires");
    foreach (DataRow row in rows)
        dtEvenements.Rows.Add(row["dateJ"], row["commentaires"]);
    indexEvenement = 0;
    AfficherEvenement();
}
```

■ Affichage par index (aucun accès BDD)

```
private void AfficherEvenement()
{
    if (dtEvenements.Rows.Count == 0) {
        lblDateEvenement.Text = "Aucun événement"; lblCompteurPages.Text = "0 / 0"; return;
    }
    DataRow row = dtEvenements.Rows[indexEvenement]; // accès direct par index
    lblDateEvenement.Text = Convert.ToDateTime(row["dateJ"]).ToString("dd/MM/yyyy");
    lblEvenement.Text = row["commentaires"].ToString();
    lblCompteurPages.Text = $"{indexEvenement+1} / {dtEvenements.Rows.Count}";
}
```

■ Boutons de navigation << < > >>

```
private void btToutDebut_Click(object sender, EventArgs e)
{ indexEvenement = 0; AfficherEvenement(); }
private void btRevenir1foisEnArriere_Click(object sender, EventArgs e)
{ if (indexEvenement > 0) indexEvenement--; AfficherEvenement(); }
private void btSuivant_Click(object sender, EventArgs e)
{ if (indexEvenement < dtEvenements.Rows.Count-1) indexEvenement++;
  AfficherEvenement(); }
private void btAllerToutAlaFin_Click(object sender, EventArgs e)
{ indexEvenement = dtEvenements.Rows.Count-1; AfficherEvenement(); }
```